

Full CRUD with Streamlit + FastAPI

A Task Manager app — code walkthrough for learners



Create



Read



Update



Delete

Reference deck — keep this handy while working through `backend_main.py` and `frontend_app.py`

User Registration Screen – Streamlit FE

Screenshot

User Registration

Frontend built with Streamlit, talking to your FastAPI backend.

Add a New User

Name

e.g. Priya

Age

1

-

+

Email

e.g. priya@example.com

Register User

All Registered Users

 Refresh User List

ID	Name	Age	Email
1	Alice	25	alice@example.com
2	Kuldeep	45	kuldeep@example.com
4	Veena	42	veena@example.com

User Registration Screen – Streamlit FE

Screenshot

Update a User

Select a user to update

1 - Alice

Name

Alice

Age

25

-

+

Email

alice@example.com

Update User

Delete a User

Select a user to delete

1 - Alice

☐ I confirm I want to delete user ID 1

Delete User

What Is CRUD?

The four operations behind almost every application that manages data.



Create

Add a brand-new record

HTTP POST



Read

Fetch one record or a list of records

HTTP GET



Update

Modify an existing record

HTTP PUT



Delete

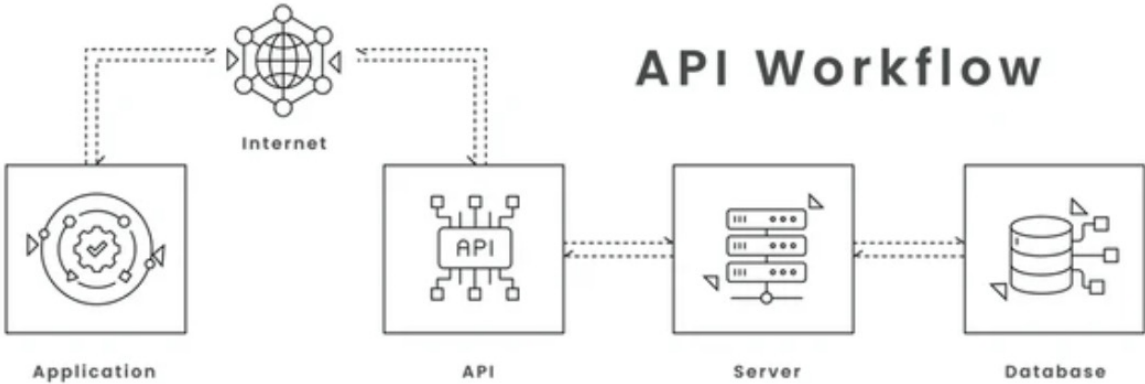
Remove a record

HTTP DELETE

What is API and REST API?



An API (Application Programming Interface) is a contract that lets two pieces of software talk to each other. Think of it like a restaurant menu: you (the client) don't need to know how the kitchen (the server) cooks the food — you just order from the menu (the API) and get your dish (the response).



A REST API (Representational State Transfer API) enables communication between client and server over HTTP.

1. It exchanges data typically in JSON format using standard web protocols.
2. Uses HTTP methods like GET, POST, PUT, PATCH, and DELETE.
3. Client sends requests to server endpoints (URLs).
4. Server returns responses such as JSON, XML, HTML, or images.
5. Maps HTTP methods to CRUD operations (Create, Read, Update, Delete).

Method	Purpose	Example
GET	Retrieve data	Get list of users: GET /users
POST	Create new data	Add a new user: POST /users
PUT	Update/replace existing data	Update user 5: PUT /users/5
PATCH	Partially update data	Update just the email of user 5
DELETE	Remove data	Delete user 5: DELETE /users/5

REST API Model



How the Two Pieces Fit Together

Two separate Python processes, talking over HTTP



The program will use MySQL DB `users.db` for all the operations. This can be Swapped with other DBs as well which support CRUD operations.

2 Terminals are needed... in one terminal the backend program will be running and in the other the front-end program will be active

```
Terminal 1: uvicorn backend_main:app --reload
```

```
Terminal 2: streamlit run frontend_app.py
```

Backend: Defining DB and Table



```
# =====  
# Connect to SQLite Database  
# (Creates a file 'users.db' if it doesn't exist)  
# =====  
conn = sqlite3.connect("users.db", check_same_thread=False)  
cursor = conn.cursor()  
  
# =====  
# Create table for users (if not already present)  
# =====  
cursor.execute('''  
CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT,  
name TEXT NOT NULL, age INTEGER NOT NULL, email TEXT UNIQUE NOT NULL)  
''')  
conn.commit()
```

Connect to SQLite Database

This database comes pre-installed with python

DB Name: users.db

We are connecting to DB name user.db

Table Name: users

The DB has a users table which has 3 columns : id, name, age and email

Task

Our task is to use CRUD operations using Streamlit FE and Flask API app

Backend: Validate the User details



```
def validate_user_data_python(name, age, email):
    if not name or len(name) < 2:
        raise ValueError("Name must be at least 2 characters long.")
    if not isinstance(age, int) or age <= 0:
        raise ValueError("Age must be a positive integer.")
    if "@" not in email or "." not in email:
        raise ValueError("Invalid email format.")
    return True

=====

class User(BaseModel):
    name: str = Field(..., min_length=2, description="User's name must be at
least 2 characters long")
    age: int = Field(..., gt=0, description="Age must be a positive integer")
    email: EmailStr # Automatically validates email format
```

Validation is Happening at the below line before posting the record in DB :

```
@app.post("/register/")
def register_user(user: User): # 'user' will be auto-validated by Pydantic
```

Function : validate_user_data_python

This is the manual way of validating the details of the user entered on the screen

Pydantic User Model: User(BaseModel)

The moment FastAPI sees user: User in the function signature, it validates the incoming JSON against the User model before your function body even starts executing. If name is too short, age isn't a positive int, or email isn't a valid email — FastAPI/Pydantic rejects the request with a 422 Unprocessable Entity automatically, and register_user() never even runs.

POST/user — Create

Backend · CREATE



```
@app.post("/register/")
def register_user(user: User): # 'user' will be auto-validated by Pydantic
    """
    Registers a new user after validation.
    """
    # Option 1: Python validation (not needed, just for comparison)
    try:
        validate_user_data_python(user.name, user.age, user.email)
    except ValueError as e:
        # Normally we won't need this once Pydantic is in place
        raise HTTPException(status_code=400, detail=f"Python Validation Failed:{e}")

    # Option 2: Insert validated data into SQLite
    try:
        cursor.execute(
            "INSERT INTO users (name, age, email) VALUES (?, ?, ?)",
            (user.name, user.age, user.email)
        )
        conn.commit()
    except sqlite3.IntegrityError:
        raise HTTPException(status_code=400, detail="Email already exists!")

    return {"message": "User registered successfully!", "user": user}
```

- task: registerUser → Create a user in the users.db DB
- IUser will be validated using Pydantic Model and manual function as well
- If Pydantic receives Bad input → automatic 422 error is invoked
- If No error than record is added the DB
- Message is returned to the FE Streamlit using the return fn which passes a dictionary back to the FE

GET /users — Read

Backend · READ



```
@app.get("/users/")
def get_all_users():
    cursor.execute("SELECT id, name, age, email FROM users")
    users = cursor.fetchall()
    if users is None:
        raise HTTPException(status_code=404, detail="Users not found!")
    return {"users": users}

=====

@app.get("/users/{user_id}")
def get_user(user_id: int):
    cursor.execute("SELECT id, name, age, email FROM users WHERE id = ?", (user_id,))
    user = cursor.fetchone()
    if user is None:
        raise HTTPException(status_code=404, detail="User not found!")
    return {"id": user[0], "name": user[1], "age": user[2], "email": user[3]}
```

- Read all records from the users table and display on the screen
- If no users found the raise error
- -
- The other function is to retrieve one record from the table
- If the record does no exists then it will give 404 error

PUT /user — Update

Backend · UPDATE



```
@app.put("/users/{user_id}")
def update_user(user_id: int, user: User):
    # 1. Make sure the user actually exists before trying to update it
    cursor.execute("SELECT id FROM users WHERE id = ?", (user_id,))
    existing = cursor.fetchone()
    if existing is None:
        raise HTTPException(status_code=404, detail="User not found!")

    # 2. Attempt the update — email is UNIQUE, so updating to an email
    # that belongs to another user will raise sqlite3.IntegrityError
    try:
        cursor.execute(
            "UPDATE users SET name = ?, age = ?, email = ? WHERE id = ?",
            (user.name, user.age, user.email, user_id)
        )
        conn.commit()
    except sqlite3.IntegrityError:
        raise HTTPException(status_code=400, detail="Email already in use
by another user!")

    return {"message": "User updated successfully!", "user": user}
```

- This function is suppose to update the users if it exists
- If the user is not found then 404 error is invoked
- If the user exists then because the fn has **user: User** as input argument pydantic will validate the user first
- Once the values passed to the fn are validated by pydantic the name, age and email address will be updated in the table
- Same 404 pattern as READ for a missing user id

DELETE /user/{id} — Delete

Backend · DELETE



```
@app.delete("/users/{user_id}")
def delete_user(user_id: int):
    # 1. Make sure the user actually exists before trying to delete it
    cursor.execute("SELECT id FROM users WHERE id = ?", (user_id,))
    existing = cursor.fetchone()
    if existing is None:
        raise HTTPException(status_code=404, detail="User not found!")

    # 2. Delete the row
    cursor.execute("DELETE FROM users WHERE id = ?", (user_id,))
    conn.commit()

    return {"message": f"User with id {user_id} deleted successfully!"}
```

- status_code=404 = 'No User id found'
- A successful delete has nothing to return
- The user is deleted from the user table
- Message is returned to the FE Streamlit using the return fn which passes a dictionary back to the FE
- This completes all 4 CRUD endpoints!

Frontend: One Function per CRUD Operation



Mental model: the frontend NEVER touches data directly — every action is an HTTP call to FastAPI.
It connects to the URL : API_BASE_URL = <http://127.0.0.1:8000> on which the FASTAPI is running



+ Add a New User

```
response = requests.post(f"{API_BASE_URL}/register/", json=payload)
```



📄 View All All Registered Users

```
response = requests.get(f"{API_BASE_URL}/users/")
```



✎ Update a User

```
response = requests.put(f"{API_BASE_URL}/users/{selected_id}", json=payload)
```



🗑 Delete a User

```
response = requests.delete(f"{API_BASE_URL}/users/{selected_id_delete}")
```

Run It Yourself

```
# Terminal 1 – start the backend first
cd backend && uvicorn backend_main:app --reload

# Terminal 2 – then start the frontend
cd frontend && streamlit run frontend_app.py
```



Create

POST + Pydantic validation + server-assigned id



Read

GET with optional query-param filtering



Update

PUT + exclude_unset for a true partial update



Delete

DELETE + 204 No Content on success

Try this: open <http://127.0.0.1:8000/docs> alongside the Streamlit app, and watch a new request appear in the Swagger UI every time you click Create/Update/Delete in Streamlit.